SPLITXPENCE

Building a Smarter Group Expense Sharing System

A portfolio case study on combining Java, Spring Boot, Hibernate, MySQL and a modern web interface into one practical expense-sharing application.

Java 17	Spring Boot	Hibernate	MySQL	JavaScript	Git

Tushar Thakur

MCA Data Science · Chandigarh University

Project Type Java Mini Project	**Interface** Web Application	**Database** MySQL	**Launch** Windows BAT / JAR
Backend Spring Boot / MVC	**Persistence** JPA / Hibernate	**Core Logic** Splits / Balances	**Frontend** HTML / CSS / JS

PROJECT STORY

Why I Built SplitXpence

I wanted to build an expense-sharing project that felt simple enough for real use but complete enough to demonstrate full-stack Java development. SplitXpence brings groups, friends, expenses, equal splitting, balances, settlements and reports into one connected workflow.

The project became an exercise in connecting backend logic with a clean user experience. An expense should update individual shares, those shares should affect balances, settlements should reduce debt, and the dashboard and reports should reflect the same database state.

CONTEXT

The Problem

Shared expense confusion

Group expenses are easy to record informally, but tracking who paid, who participated, what each person owes and what has already been settled quickly becomes confusing.

What SplitXpence does

Brings groups, shared expenses, equal splits, balances, settlements and reports into one web workflow backed by a relational MySQL database.

KEY CAPABILITIES

What the Application Can Do

Groups & friends

Create groups, maintain a friend list, add members, edit group details and manage shared-expense communities.

Expenses & splits

Record expenses, select participants, assign the payer, categorize transactions and create equal split records automatically.

Balances & settlements

Calculate group balances, view personal balance, simplify who owes whom and record full or partial settlements.

Dashboard & search

See totals and recent expenses while searching transactions by title, group, category and date range.

History & reports

Review expense and settlement history and generate group, category and monthly spending summaries.

Profile & interface

Manage account details and friends through a responsive colorful interface with modal-based results and quick actions.

ARCHITECTURE

How SplitXpence Is Structured

SplitXpence follows a compact web architecture: HTML, CSS and JavaScript provide the interface, Spring MVC controllers handle requests, Spring Data JPA repositories connect business actions to Hibernate, and MySQL stores users, groups, expenses, splits and settlements.



Layer	Main Technology	Responsibility
Interface	HTML / CSS / JavaScript	Responsive pages, forms, navigation, Fetch requests, tables and modal feedback.
Application	Spring Boot / Spring MVC	Session handling, validation, workflows, expense logic, balances and HTTP endpoints.
Data & persistence	Spring Data JPA + Hibernate + MySQL	Entity mapping, CRUD operations, relationships, queries and persistent transaction data.

EXAMPLE WORKFLOW

Data Flow: Adding a Shared Expense

Select group → **Enter expense** → **Choose payer** → **Select members** → **Create splits** → **Recalculate balance** → **Show result**

A logged-in user selects a group, enters the expense details, chooses the payer and participants, and submits the form. The backend validates group access, stores the expense, creates equal split records for the selected participants, and makes the updated balances immediately available to the dashboard, reports and settlement logic.

CORE RELATIONSHIPS

Relational Data Model

Entity	Connection	Usage
User	Friends / groups / payments	Authentication, profile and transaction ownership
ExpenseGroup	Creator / members	Container for expenses, balances and settlements
Expense	Group / payer / splits	Shared transaction amount, date and category
Settlement	Group / paid from / paid to	Reduces outstanding debt between members

DATA ENGINEERING

The Data Work Behind the UI

SplitXpence uses a relational MySQL model with focused JPA entities and repositories. Expense, split and settlement records are stored separately so the application can calculate balances accurately while keeping CRUD operations and reporting queries straightforward.

Object	Examples	Purpose
JPA entities	User, Friend, ExpenseGroup, Expense, ExpenseSplit, Settlement	Represent relational business data as Java objects.
Repositories	find by email, group, category, title and date	Keep database access reusable and concise using Spring Data JPA.
Expense splits	participant + share amount	Store the exact equal share assigned to each selected participant.
Reports & balance logic	group totals, categories, months, debt matching	Convert stored transactions into balances, summaries and settlement suggestions.

RELIABILITY

State & Data Safety

SESSION LOGIN	ACCESS CHECKS	RELATIONAL DATA	SAFE CONFIG
Track the active user	Validate group actions	Keep splits separate	Ignore local DB password

ENGINEERING CHOICES

A Few Design Decisions I’m Proud Of

Equal split logic

When an expense is created, the selected participants receive equal share records. This keeps the calculation model transparent and makes later balance computation easy to explain.

Session-based workflow

Login stores the active user in the HTTP session, allowing protected pages and controllers to identify the current user without adding unnecessary complexity for a college mini-project.

Layered persistence

Spring Data JPA repositories keep database access concise while Hibernate maps Java entities to MySQL tables and manages routine persistence operations.

Consistent frontend feedback

Fetch-based form handling and reusable modal results keep users inside the application instead of sending them to raw JSON or plain-text response pages.

SCOPE

What the Project Includes - and What It Does Not

Included

✓ Registration, login, logout and profile

✓ Friend list and group member management

✓ Group create, edit, select and protected delete

✓ Expense CRUD with categories and participant selection

✓ Equal expense splitting

✓ Group balance, personal balance and who-owes-whom logic

✓ Full and partial settlement tracking

✓ Dashboard, search, history and reports

✓ Responsive HTML / CSS / JavaScript interface

✓ MySQL persistence and Windows BAT launcher

Intentionally out of scope

– Custom percentage or unequal split rules

– Spring Security / production authentication

– Cloud deployment or mobile application

– Receipt scanning or file attachments

– Multi-currency conversion

– Email invitations or notifications

– Online payment gateway integration

– Enterprise roles and administration

REFLECTION

What I Learned

SplitXpence helped me connect Java OOP, Spring Boot, HTTP requests, relational data modelling and frontend interaction in one project. The most useful lessons came from understanding how one action, such as adding an expense or recording a settlement, affects multiple parts of the application.

Spring web architecture
Controllers, sessions and HTTP flows.

MySQL + JPA
Entities, repositories and CRUD.

Expense logic
Splits, balances and settlements.

Frontend integration
Fetch API, forms and modals.

Data modelling
Groups, expenses and splits.

Project delivery
Git, testing and documentation.

CONCLUSION

Final Thoughts

SplitXpence began as a Java mini-project, but it became a practical exercise in full-stack web application development. It combines Spring Boot, MVC controllers, JPA repositories, Hibernate ORM, MySQL and a responsive browser interface without adding unnecessary enterprise complexity.

For me, the strongest part of the project is the connected money flow: an expense creates participant shares, those shares affect balances, settlements adjust what people owe, and the same data appears on the dashboard and in reports. That end-to-end flow is what makes SplitXpence feel like a complete application rather than a set of isolated CRUD screens.

EXPLORE

Project Links

GitHub	https://github.com/tushar-011
LinkedIn	https://www.linkedin.com/in/tushar-thakur-8848a7396/
Email	artificial.thakur@gmail.com